

eAI Application Overview

80-78521-8 Rev. AA

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision History

Revision	Date	Description
AA	June 2024	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyungchul.won@lge.com

Contents

1 eAI Overview	<u>4</u>
2 eAI Simulator	<u>12</u>
3 How to Integrate eAI Model into qsh-camera	<u>16</u>
4 eAI Model Performance and Memory Allocation Review	<u>21</u>
5 How to Raised a Ticket to the eAI Team	<u>25</u>
6 References	<u>27</u>

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyungchul.won@lge.com



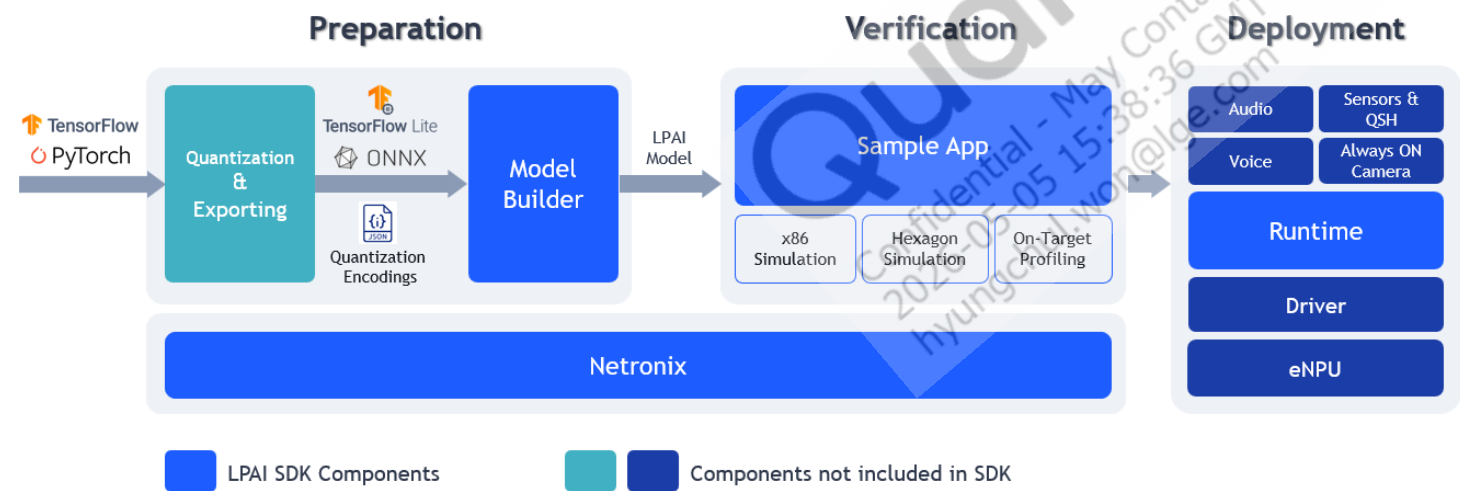
Section 1

eAI Overview

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyungchul.won@lge.com

eAI SDK

- eAI, also known as LPAI, Qualcomm Low Power Artificial Intelligence SW SDK assists developers to offload models on eNPU.
- QPM client is the only distribution channel of LPAI SDK.
- eAI developers also need to manually install some dependencies:
 - Hexagon SDK 5.x , 5.5.0.1
 - Hexagon SDK early release, 4.0.1.0
- SDK overview



Qualcomm Hexagon SDK Products	
Q	Audio1x
	Compute_AddOn1x
Q	eAI_Early_Release
Q	Hexagon SDK 6.x
Q	HexagonSDK_Early_Release
	Hexagon™ SDK 4.x
Q	Hexagon™ SDK 5.x
Q	Linux Red Hat® Addon 1.x
Q	Linux Vehicle Addon 1.x
Q	Linux_Android_GVM Addon 1.x
Q	Low Power Artificial Intelligence (LPAI)
Q	QNX Addon 1.x
Q	Qualcomm® Hexagon™ IDE
Q	Windows on Snapdragon Addon 1.x

eNPU Evolution

- eNPU is the final entity for eAI implementation on Qualcomm platform. eNPU core is supported from SM8350 or later.

	SM8350 (eNPU1)	SM8450 (eNPU2)	SM8550 (2x eNPU3)	SM8650 (2x eNPU4)	SM8750 (2x eNPU5)
MACs (8x8)	128 MAC	128 MAC	128 MAC+128MAC	512MAC+256MAC	1024MAC + 256MAC
Operators	GEMM, GEMV	CNN, RNN	Attention, SVDF	/	Layernorm, softmax
Additional features	/	Model decompression (20-30% bandwidth and memory footprint reduction)	/	New compression format(BFMC)	Runtime Sparse Activation compression
Precision	16-bit Int, 8-bit Int	16-bit Int, 8-bit Int	16-bit Int, 8-bit Int	16-bit Int, 8-bit Int, 4-bit int(weight)	4w4a, 4w8a, 4w16a, 8w8a, 8w16a, 16w16a
Runtimes	eAI	eAI, low-level Ops	eAI, low-level Ops	eAI, low-level Ops	eAI, low-level Ops

eAI Tools

■ ONNX Rectifier

- Tool used to pre-process, simplify, and clean-up ONNX models prior to conversion for eAI execution. Output is a simplified *.onnx* model. Recommended but optional step.

■ ONNX Executor

- Tool approximates the eAI runtime behavior in floating point ONNX Runtime with the objective of validating eAI models and their outputs.

■ Data Relayout

- Tool used to convert between different layouts with given shape and data.

■ eAI Model Builder

- Tool that converts *.onnx* or *.tflite* models to eAI model definition format (*.eai*). The conversion step is a mandatory prerequisite prior to model execution.

■ eAI Model Visualizer

- Netronix Model Visualizer provides a GUI based viewer for *.onnx*, *tflite*, and *.eai* converted models. Experimental new features include a built-in model converter that may be used in place of eAI Model Builder, a generation of quantization JSON file, and a built-in model execution runtime.

■ eAI Quantize

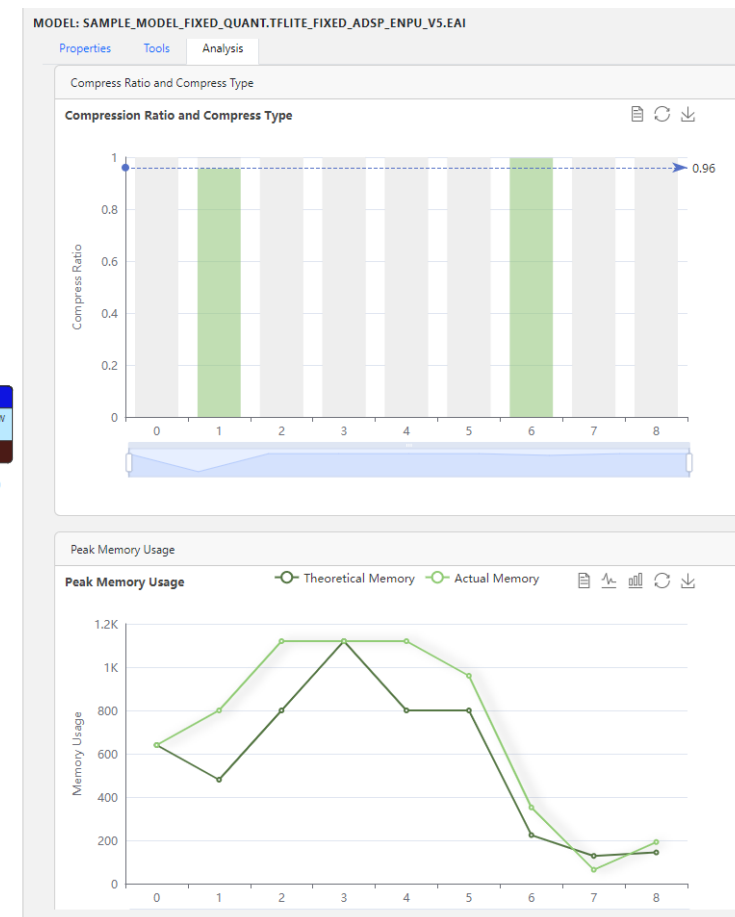
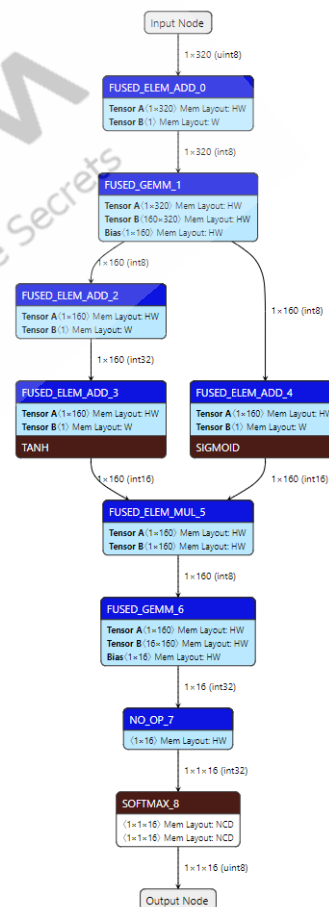
- Tool used to quantize or de-quantize tensors for a given eAI model, using the quantization information stored in the model as default.

Netronix for eAI Model

- Based on the open-source neural network visualizer Netron, Qualcomm has developed Netronix, a one-stop toolchain for eAI model. This tool has been shared through LPAI SDK at `<Hexagon_SDK>/addon/LPAI/eNPU5/4.8.0/netronix/`

- Netronix key features:

- Present and visualize model (ONNX, TFLite, eAI, and so on.) topology
- Dummy quantization to generate JSON encodings and dummy input data for float ONNX model
- Quantize float ONNX model with AIMET via Post Training Quantization schemes to compute the encodings
- Convert float ONNX or TFLite model to eAI model
- Execute float or fixed-point eAI model with the built-in simulation environment
- Model Metadata Per-layer Dump and present eAI model metadata in interactive charts



eAI Runtime

- The eAI framework is designed for deeply embedded environments.
 - Low memory footprint
 - Support for ADSP's Low Power Island
 - Operates directly on DSP without any need for CPU interventions
- One of the design goals of eAI is to use the least amount of memory within and between various use cases. One of the aspects is the ability to reuse runtime scratch memory buffers between different instances of the eAI runtime.
- Examples of such scenarios are executing different models or multiple instances of the same model to process different audio channels, with each instance of the models utilizing a different instance of the eAI runtime.
- The following features are supported:
 - Fixed point engine
 - 8 bit, 16 bit and 32 bit tensors
 - eNPU HW accelerator offloading
 - Low Power Island Support
 - Ability to share scratch memory between multiple instances of eAI
 - eNPU5 supports 4 weights for convolution.

eAI Memory

■ Scratch Memory

- This is the memory pool to hold intermediate results during network execution which can be reused (overwritten during network execution). The user is responsible for allocating this memory pool by querying the eAI runtime for the scratch memory requirements for their particular model and passing it into the eAI runtime.

■ Persistent Memory

- This is the memory pool to hold intermediate results during network execution which can't be reused (persistent during network execution). An example of this is in the RNN op; the tensors for holding the previous state will be persistent. The user is responsible for allocating this memory pool by querying the eAI runtime for the persistent memory requirements for their particular model and passing it into the eAI runtime.

■ IO Memory

- This is the memory pool(s) containing the input and output tensors. This can be user provided or planned into the scratch memory pool. By default the input and output tensors will be planned into the scratch memory pool. If the user is providing the input/output buffer, the starting address needs to be aligned correctly before passing to eAI runtime. The alignment requirement of each tensor can be viewed in Netronix and queried using `eai_get_property` API call and eAI property `EAI_PROP_TENSOR_INFO`.

format	eAI v4.7.0
name	sample_model_fixed.eai
description	converted from ONNX using eAI model builder v4.7
Not Offloaded Nodes	NO_OP_0
flags	COMPRESSED FIXED_MODEL FALLBACK_DISABLED 0x1
graph_size	54812
Persistent Mem Size	0
Scratch Mem Size	640
Enpu Version	v5
Model Target	Hexagon DSP
Graph Total:	Count: 11, Mac Count: 54240
FUSED_ELEM_ADD	Count: 2, Mac Count: 320, Not Offloaded: 0
FUSED_ELEM_MUL	Count: 1, Mac Count: 160, Not Offloaded: 0
FUSED_GEMM	Count: 2, Mac Count: 53760, Not Offloaded: 0
NO_OP	Count: 5, Mac Count: 0, Not Offloaded: 1
SOFTMAX	Count: 1, Mac Count: 0, Not Offloaded: 1

eAI API Updates

■ Deprecated result code

- EAI_HAS_MORE

■ Deprecated eAI Property

- EAI_PROP_MLA_USAGE
- EAI_PROP_RESET_LAYER
- EAI_PROP_ALIGNMENT
- EAI_PROP_TENSOR_SIZE_INFO
- EAI_PROP_MLA_OFFLOAD_SUMMARY

■ Deprecated API functions

- eai_init_ex()
- eai_preapply()

■ New APIs

- eai_init()
- EAI_PROP_PERSISTENT_MEM
- EAI_PROP_MODEL_ALIGNMENT

■ EAI client need to get and allocate persistent memory

```
// Get persistent buffer info
eai_ret = eai_get_property(context->eai_handle, EAI_PROP_PERSISTENT_MEM, &persistent_memory);
if (eai_ret != EAI_SUCCESS) {
    printf("eai_get_property(EAI_PROP_PERSISTENT_MEM) FAIL. result = %d\n", eai_ret);
    return -1;
}

context->persistent_buffer_size = persistent_memory.memory_size;
context->persistent_buffer = (uint8_t *)malloc(persistent_memory.memory_size);
LOG_AND_RETURN_IF_TRUE(!context->persistent_buffer, SAMPLE_APP_FAIL);

persistent_memory.addr = context->persistent_buffer;
persistent_memory.memory_type = PERSISTENT_BUFFER_MEM_TYPE;
// Set persistent buffer for eai api
eai_ret = eai_set_property(context->eai_handle, EAI_PROP_PERSISTENT_MEM, &persistent_memory);
LOG_AND_RETURN_IF_TRUE(eai_ret != EAI_SUCCESS, SAMPLE_APP_FAIL);
```

■ EAI client need to get and check IO tensor memory alignment

```
// Get model I/O tensor information and allocate them
SAMPLE_APP_RESULT get_model_tensor_info(struct eai_sample_lite_context *context) {
    EAI_RESULT eai_ret = EAI_SUCCESS;

    for (int i = 0; i < 2 && eai_ret == EAI_SUCCESS; i++) {
        eai_ports_info_t ports_info;
        ports_info.input_or_output = i;
        eai_ret = eai_get_property(context->eai_handle, EAI_PROP_PORTS_NUM, &ports_info);
        LOG_AND_RETURN_IF_TRUE(eai_ret != EAI_SUCCESS, SAMPLE_APP_FAIL);

        context->tensor_count[i] = ports_info.size;
        context->tensors[i] = malloc(sizeof(eai_tensor_info_t) * ports_info.size);
        LOG_AND_RETURN_IF_TRUE(context->tensors[i] == NULL, SAMPLE_APP_FAIL);

        for (unsigned int j = 0; j < ports_info.size; j++) {
            context->tensors[i][j].index = j;
            context->tensors[i][j].input_or_output = i;

            eai_ret = eai_get_property(context->eai_handle, EAI_PROP_TENSOR_INFO, &(context->tensors[i][j]));
            LOG_AND_RETURN_IF_TRUE(eai_ret != EAI_SUCCESS, SAMPLE_APP_FAIL);
        }
    }

    return SAMPLE_APP_SUCCESS;
}
```



Section 2

eAI Simulator

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyunchul.won@lge.com

eAI Sample Application Lite for Offline Simulation

■ eAI provides alternative simulators based on `eai_sample_app_lite` for different environments:

□ Build instructions for specific environment:

```
./build.sh --build_linux --clean_build --build_enpu_ver 5 //for linux environment
```

```
./build.sh --build_hexagon_sim --clean_build --build_enpu_ver 5 //for hexagon simulation
```

```
./build.sh --build_adsp --clean_build --build_enpu_ver 5 //for adsp environment
```

□ Execution instructions for specific environment:

• For linux environment:

```
../build-fixed32/example/eai_sample_app_lite/eai_sample_app_lite
m ../sample_models/fixed32/model1/model/sample_model_fixed.eai
i ../sample_models/fixed32/model1/input/sample_model_input_fixed.raw
o ../sample_models/fixed32/model1/output/
```

• For Hexagon simulation :

```
${DEFAULT_HEXAGON_TOOLS_ROOT}/Tools/bin/hexagon-sim --simulated_returnval ../binaries/hexagon_sim/enpu_driver/runelf.pbn
--rtos ./osam.cfg --cosim_file ../binaries/hexagon_sim/enpu_cosim/q6ss.cfg --nullptr=2 --dsp_clock 500 -mv79m_1
--l2cfg_base 0x047A --subsystem_base 0xFC90 -- ../build-fixed32/example/eai_sample_app/eai_sample_app_lite
-m ../sample_models/fixed32/model1/model/sample_model_fixed.eai
-i ../sample_models/fixed32/model1/input/sample_model_input_fixed.raw -o ../sample_models/fixed32/model1/output/
```

Enpu_Launcher for On-Device Simulation

- Enpu_launcher is a skeleton app (RPC client) running on the android/ARM host side, and it talks to the components in the ADSP through the RPC call. This tool helps customer to test pre-deployed model on the target platform more conveniently.
- How to execute enpu_launcher on devices
 - sign device by serial_number
 - `adb shell cat /sys/devices/soc0/serial_number` (Ex: 0x83BA6DB5)
 - `python %HEXAGON_SDK_ROOT%/tools/elfsigner/elfsigner.py -t 0x83BA6DB5`
 - `adb push testsig-0x83ba6db5.so /vendor/lib/rfsa/adsp`
 - push enpu_launcher to device
 - `cd %HEXAGON_SDK_ROOT%/addons/LPAI/eNPU5/$VERSION/binaries/adsp/enpu_launcher`
 - `adb push enpu_launcher /vendor/bin`
 - `adb push libenpu_launcher_skel.so /vendor/lib/rfsa/adsp`
 - push eai_sample_app, models and run enpu_launcher on device
 - `adb push build-fixed32/example/eai_sample_app/libeai_sample_app_lite.so /vendor/lib/rfsa/adsp`
 - `adb push $HEXAGON_SDK_ROOT/addons/LPAI/eNPU5/$VERSION/example/sample_models/fixed32/model1/ /data/local/tmp`
 - `adb shell "export ADSP_LIBRARY_PATH=/vendor/lib/rfsa/adsp; enpu_launcher libeai_sample_app_lite.so -m /data/local/tmp/model1/model/sample_model_fixed.eai -i /data/local/tmp/model1/input/sample_model_input_fixed.raw -o /data/local/tmp/model1/output"`

eAI Runtime Statistics

- Enpu_launcher provides various statistics for eAI model, especially `-disable_linking` option decouples latency of fused nodes.

- Performance stats can be found under `[model_name]_perf_layer_stats.csv`
- Memory stats can be found under `[model_name]_mem_info.csv`
- Profiling stats can be found under `[model_name]_perf_info.csv`

- The following stats come from per-layer stats. We can find that all computing layers are offloaded on eNPU and we can also

Index	Layer_type	HW_Offloaded	Offload_Merge_Group	Offload_count	Exec Count	Last Exec Time	Avg Time	Total Time
0	TRANSPOSE	No	-	-	1	469 us	469 us	469 us
1	SUPERCONVOLUTION	Offloaded	1	1	1	195030 us (1)	195030 us (1)	195030 us (1)
2	MAX_POOL	Offloaded	2	1	1	3222 us (2)	3222 us (2)	3222 us (2)
3	SUPERCONVOLUTION	Offloaded	3	1	1	137869 us (3)	137869 us (3)	137869 us (3)
4	SUPERCONVOLUTION	Offloaded	4	1	1	33651 us (4)	33651 us (4)	33651 us (4)
5	SUPERCONVOLUTION	Offloaded	5	1	1	226160 us (5)	226160 us (5)	226160 us (5)
6	SUPERCONVOLUTION	Offloaded	6	1	1	210000 us (6)	210000 us (6)	210000 us (6)
7	SUPERCONVOLUTION	Offloaded	7	1	1	27433 us (7)	27433 us (7)	27433 us (7)
8	SUPERCONVOLUTION	Offloaded	8	1	1	212831 us (8)	212831 us (8)	212831 us (8)
9	SUPERCONVOLUTION	Offloaded	9	1	1	206284 us (9)	206284 us (9)	206284 us (9)
10	SUPERCONVOLUTION	Offloaded	10	1	1	24945 us (10)	24945 us (10)	24945 us (10)
11	SUPERCONVOLUTION	Offloaded	11	1	1	207072 us (11)	207072 us (11)	207072 us (11)
12	SUPERCONVOLUTION	Offloaded	12	1	1	25380 us (12)	25380 us (12)	25380 us (12)
13	SUPERCONVOLUTION	Offloaded	13	1	1	207971 us (13)	207971 us (13)	207971 us (13)
14	SUPERCONVOLUTION	Offloaded	14	1	1	25297 us (14)	25297 us (14)	25297 us (14)
15	SUPERCONVOLUTION	Offloaded	15	1	1	207143 us (15)	207143 us (15)	207143 us (15)
16	SUPERCONVOLUTION	Offloaded	16	1	1	25375 us (16)	25375 us (16)	25375 us (16)
17	SUPERCONVOLUTION	Offloaded	17	1	1	206371 us (17)	206371 us (17)	206371 us (17)



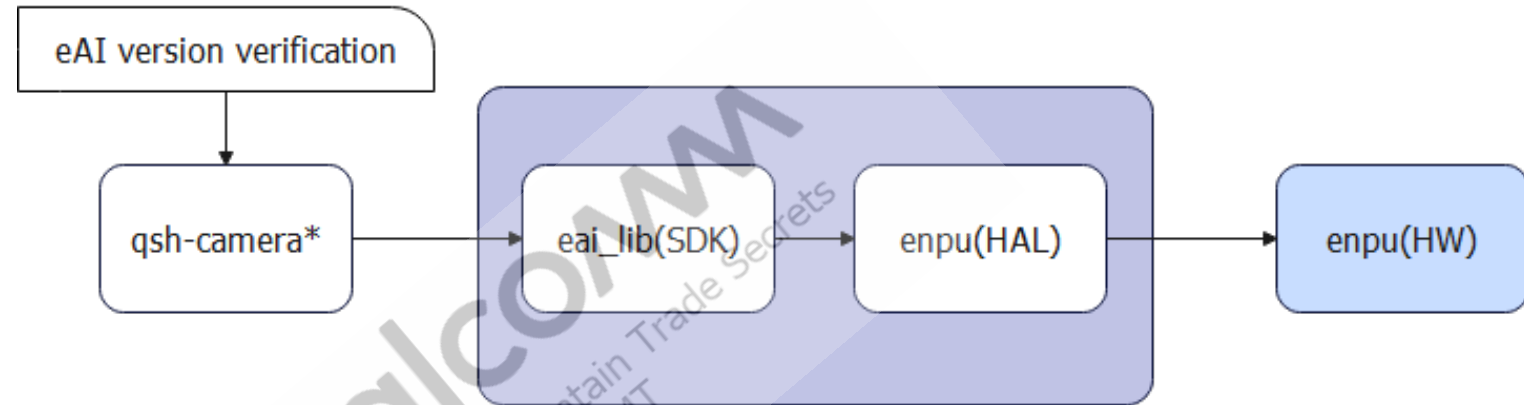
Section 3

How to Integrate eAI Model into qsh-camera

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyungchul.won@lge.com

Source Tree for aDSP Source Code

```
|-- avs
|-- build
|-- config
|-- core
|-- crashman
|-- dsp_kailua.adsp.coreQ.elf
|-- dsp_kailua.adsp.core_questQ.elf
|-- dsp_kailua.adsp.prodQ.elf
|-- eai_lib
|-- enpu
|-- fastrpc_tests
|-- gpr
|-- image_info.json
|-- loc
|-- performance
|-- platform
|-- qdsp6
|-- qmimsgs
|-- qsh
|-- sdc_core
|-- sdc_sensors
|-- ssc
|-- ssc_api
|-- ssc_drivers
|-- testframework
|-- tools
`-- ultrasound
```



qsh-camera: Qualcomm Sensing Hub Camera

eAI Model to .h File

Qualcomm provides tools named as `tensor2h.py` which can help to convert eAI model .h file and then integrate into qsh-camera module folder.

■ Source tree for qsh-camera-oem1 src directory.

```
-- src
|-- qsh_camera_oem1.c
|-- qsh_camera_oem1_eai.c
|-- qsh_camera_oem1_eai.h
|-- qsh_camera_oem1_eai_island.c
|-- qsh_camera_oem1_eai_model.h
|-- qsh_camera_oem1_eai_private.h
|-- qsh_camera_oem1_eai_stub.c
|-- qsh_camera_oem1_eai_stub.h
|-- qsh_camera_oem1_mem.c
|-- qsh_camera_oem1_mem.h
|-- qsh_camera_oem1_sensor.c
|-- qsh_camera_oem1_sensor_instance.c
|-- qsh_camera_oem1_sensor_instance_island.c
|-- qsh_camera_oem1_test_sensor.c
`-- qsh_camera_oem1_test_sensor_instance.c
```



Convert model to .eai and .h

How to Trigger the qsh-camera Implementation

There are two orientations to load the eAI model on qsh-camera.

1. Enable the debug flash on `qsh_camera_oem1/build/qsh_camera_oem1.scons`,

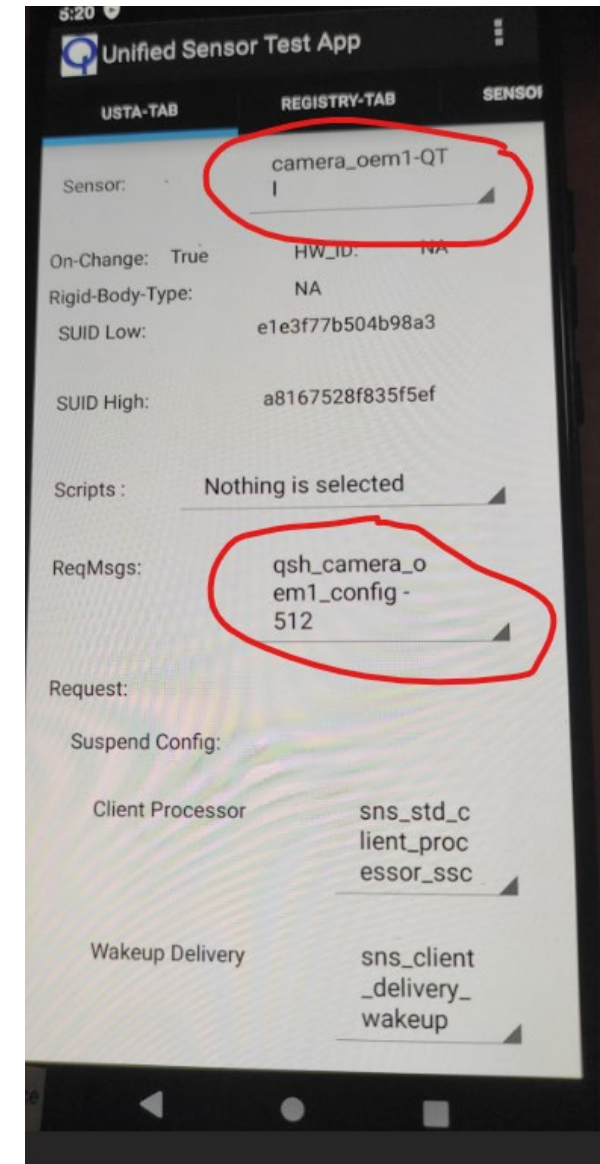
```
custom_cpp_defines = [  
    # 'ENABLE_TEST_SENSOR', # enable this flag will trigger to qsh-camera while reboot device automatically  
    # 'ENABLE_FRAME_DUMP', # this is only enabled during debugging, due to power/performance/privacy impacts  
    # 'USE_OEM1_EAI_STUB',  
]
```

2. Unified Sensor Test App:

- a. Before pressing button of **SendRequest** which help to trigger qsh-camera, we need to push `.proto` to device,

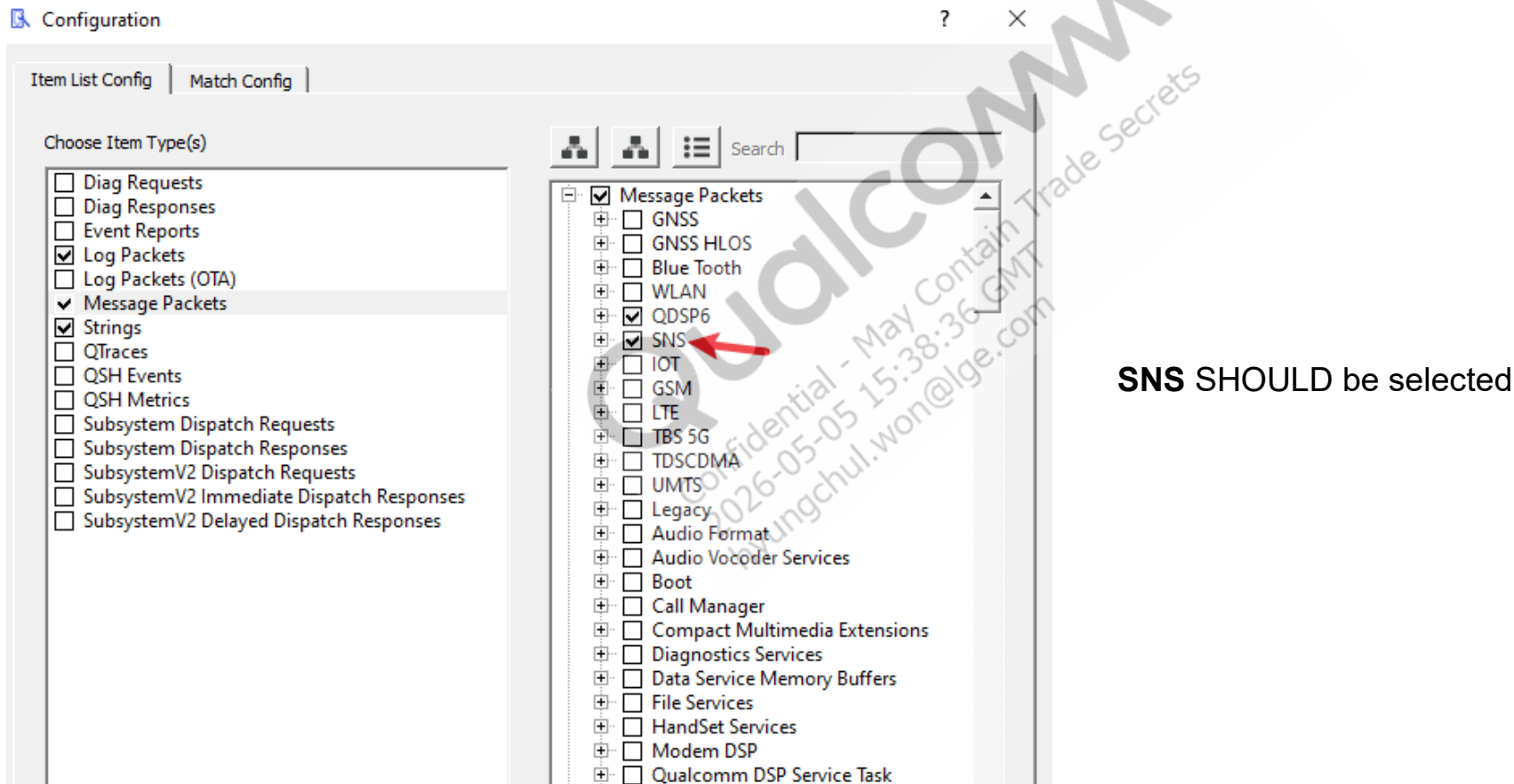
```
adb push adsp_proc\qsh\qsh_camera_oem1\pb\qsh_camera_oem1.proto /vendor/etc/sensors/proto
```

- b. Then, capture DSP logs with QXDM or mini-dm.



Capture Log for qsh-camera eAI

- This page only introduces the configuration for how to get logs for qsh-camera eAI on QXDM.
- After loading the aDSP logger settings to QXDM, users need to enter config page by “F3” command. Then, select the **SNS** option to capture the qsh-camera log to QXDM.





Section 4

eAI Model Performance and Memory Allocation Review

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyungchul.won@lge.com

eAI Model Performance Analysis

- As csim launched on x86 cannot identify the time consumption, it is recommended to measure the eAI model performance through `enpu_launcher` which works on target devices.
- Users can capture the aDSP log by QXDM while eAI model is executing `enpu_launcher`.
- Here illustrates a sample model to introduce the eAI interface time consumption in aDSP default mode.

eAI Interface	Time(μs)
eai_init	920
eai_prapply	63703
eai_apply	29835
eai_execute	48227

```
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_init
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_preapply
00000000 [ 00000000] [00000000]:00000000: ADSP: 63703 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 95402770 cycles
00000000 [ 00000000] [00000000]:00000000: ADSP: 47.70 MPPS
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_preapply
00000000 [ 00000000] [00000000]:00000000: ADSP: 63703 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 95402770 cycles
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_apply
00000000 [ 00000000] [00000000]:00000000: ADSP: 29835 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 44680473 cycles
00000000 [ 00000000] [00000000]:00000000: ADSP: 22.34 MPPS
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_apply
00000000 [ 00000000] [00000000]:00000000: ADSP: 29835 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 44680473 cycles
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_execute - overall [ 4]:
00000000 [ 00000000] [00000000]:00000000: ADSP: 192908 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 20.735273 executes/sec
00000000 [ 00000000] [00000000]:00000000: ADSP: 241101061 cycles
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_execute - overall [ 4]:
00000000 [ 00000000] [00000000]:00000000: ADSP: 192908 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 20.735273 executes/sec
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_execute - average
00000000 [ 00000000] [00000000]:00000000: ADSP: 48227 us
00000000 [ 00000000] [00000000]:00000000: ADSP: 60275265 cycles
00000000 [ 00000000] [00000000]:00000000: ADSP: 30.14 MPPS
00000000 [ 00000000] [00000000]:00000000: ADSP: eai_execute - average
```

Adjust aDSP Clock to Test Model Implementation

- aDSP clock has multiple settings according the working status as below.

```
const adspToMlFreq_arrayv1[] = {  
    // adspFreqHz,      MlFreqHz  
    { 460800000,        122880000 }, // low_svs_D1 f-max  
    { 595200000,        204800000 }, // low_svs f-max  
    { 787200000,        245760000 }, // svs ADSP f-max  
    { 960000000,        307200000 }, // svs_1  
    { 1190400000,       409600000 }, // nominal  
    { 1324800000,       409600000 }, // nominal_l1  
    { 1497600000,       614400000 }, // Turbo  
};
```

- The left values on above array can be set with tools of sysMonApp which is available on Hexagon SDK.
- Executing inline commands can help reset the aDSP power to corresponding configuration. '3000' means setting the DDR bandwidth and aDSP clock to max.

```
sysMonApp clocks set --coreClock 3000 --busClock 3000 // 1497.6 is the burst mode for aDSP clock.
```

- After above configuration, users can observe that the interface of `eai_execute` and other interface will lead to different performance.

eAI Memory Allocation

- eNPU memory definition is recorded at `config/enp_libs/custom_config.xml`. Customizing the configuration xml can reallocate the memory for eNPU.

```
<physical_pool name="CAM_LLCC_ISLAND1_POOL" island="true" collapse="tcm">
  <region base="0xCFFA40000" size="0x1C0000" cache_policy="l1_wb_l2_cacheable"/>      <!--SCID 12 - 1.75MB -->
</physical_pool>
<physical_pool name="CAM_LLCC_ISLAND4_POOL" island="true" collapse="tcm">      <!--Model header file stored at here and llc1 if not enough -->
  <region base="0xFFFC00000" size="0x40000" cache_policy="l1_wb_l2_cacheable"/>      <!--SCID 15 - 256kB -->
</physical_pool>
<physical_pool name="TCM_PHYSPOOL" island="true" collapse="tcm">      <!-- 832KB common pool, scratch memory used->
  <region base="0x06400000" size="0xD0000" name="QURTOS_ISLAND_REGION_TCM" cache_policy="writethrough"/>
</physical_pool>
<physical_pool name="AUDIO_ISLAND_TCM_PHYSPOOL" island="true" collapse="tcm">      <!-- 704KB audio only pool -->
  <region base="0x064D0000" size="0xB0000" name="QURTOS_ISLAND_REGION_TCM" cache_policy="writethrough"/>
</physical_pool>
<physical_pool name="AUDIO_ISLAND_LPASS_TCM_POOL" island="true" collapse="tcm">      <!-- 896KB Audio only pool -->
  <region base="0x06080000" size="0xE0000" cache_policy="l1_wb_l2_cacheable"/>
</physical_pool>
<physical_pool name="CAM_ISLAND_LPASS_TCM_POOL" island="true" collapse="tcm">      <!-- 0.25MB Camera only pool -->
  <region base="0x06160000" size="0x40000" cache_policy="l1_wb_l2_cacheable"/>
</physical_pool>
<physical_pool name="SSC_ISLAND_LPASS_TCM_POOL" island="true" collapse="tcm">      <!-- 1.870MB SSC only pool -->
  <region base="0x061A0000" size="0x1DF000" cache_policy="l1_wb_l2_cacheable"/>
</physical_pool>
<physical_pool name="SSC_SDC_ISLAND_LPASS_TCM_POOL" island="true" collapse="tcm">      <!-- 4KB SSC only pool -->
  <region base="0x0637F000" size="0x1000" cache_policy="uncached"/>
</physical_pool>
```



Section 5

How to Raised a Ticket to the eAI Team

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyunchul.won@lge.com

PA Rules for eAI Ticket

- Selecting the following PA will raise an eAI ticket to the Qualcomm AI team automatically.

Problem Description Info

* Problem Area 1 ⓘ

Multimedia

[View all dependencies](#)

* Problem Area 2 ⓘ

Machine Learning (AI)

[View all dependencies](#)

* Problem Area 3 ⓘ

eAI(AON)

[View all dependencies](#)



Section 6

References

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyunchul.won@lge.com

Documents and Standards

Title	Number
Qualcomm Technologies, Inc.	
eAI Model Design Guide	80-58216-4
eAI Application on AON	80-58216-3
eAI Integration in CAPI Modules	80-VN500-23
Resources	
KBA-240513235755	eAI Update on SM8750

Qualcomm
Confidential - May Contain Trade Secrets
2026-05-05 15:38:36 GMT
hyungchul.won@lge.com

Questions?

For additional information or to submit technical questions, go to: <https://www.qualcomm.com/support>

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“**Qualcomm Technologies**”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “**Qualcomm Internal Use Only**”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com or the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.